Auction Smart Contract Testing Report

This document outlines the testing results for the Smart Contract Auction specification, based on the Safety and Threat Analysis (SAA) defined in the 'Safety And Threat Analysis Tutorial' document. The objective of this testing phase is to verify whether the identified security vulnerabilities, as detailed and described within the Security Assessment Artifacts (SAAs), are indeed exploitable. This verification process utilizes a dedicated test-suite executed within a simulated environment. The simulation platform of choice is 'sc-tools', a specialized environment designed for smart contract analysis and testing. The tests are designed to mimic potential attack scenarios outlined in the SAAs, thereby validating the reported susceptibility of the smart contract logic to real-world exploits.

SAA Testing Results Summary

The following table summarizes the testing status for each identified Failure Mode/Attack (SAA) from the specification.

Reference	Failure Mode/Attack	Severity	Test Status	Comments
SAA-0001	`adHighestBid` set to `Just newBid` on deployment such that: `newBid.bAmount` > Ada at produced utxo	High	Addressed	The test passed, indicating that the contract logic did not fail, meaning the vulnerability is present in the current specification/implementation. This attack was simulated according to the failure mode when no NFT minting was associated.
SAA-0002	`adHighestBid` set to `Just newBid` on deployment such that: `newBid.bAmount` < Ada at produced utxo	Medium	Addressed	Two tests were necessary for this requirement: bidding and payout. Both tests do not confirm funds locked forever. The bidding test allowed us to place a bid, and the payout test allowed us to proceed with the payout transaction, even when adding a fake bid during deployment.
SAA-0003	`adHighestBid` set to `Just newBid` on deployment such that:	Low	Addressed	The test passed, indicating that the contract logic did not fail, meaning the vulnerability

Reference	Failure Mode/Attack	Severity	Test Status	Comments
	`newBid.bAmount` = Ada at produced utxo			is present in the current specification/implementation. This attack was simulated according to the failure mode when no NFT minting was associated.
SAA-0004	The `AuctionDatum` inline datum provided on deployment is unbounded (extra fields introduced)	High	Addressed	Two tests were necessary for this requirement: bidding and payout. Both tests do not confirm the failure, since they indicate “parsing error” when we introduce extra fields at the Builtin Data representation level.
SAA-0005	A large reference script is present at produced utxo on deployment.	High	Not Tested	This failure mode needs explicit testing to verify if the system prevents large reference scripts on deployment, potentially via a minting policy. We encountered an issue with simulation using `sc-tools`, suggesting the necessity of better understanding how the tool processes large scripts.
SAA-0006	The Auctioned Token is not present at the produced utxo upon deployment	Medium	Addressed	Three tests were necessary for this requirement: bid and payout with no auctioned token, and payout when the token is in the seller’s wallet. The first test fails when bidding with no token, as expected. The second test permits the payout even if no token is included, provided the token is already present in the seller's wallet. The final test, as anticipated, resulted in an error. This occurred when attempting to execute the

Reference	Failure Mode/Attack	Severity	Test Status	Comments
				payout without any locked tokens and with the required token absent from the seller's wallet.
SAA-0007	Other tokens (apart from the Auctioned Token and the NFT Token) are present at the produced utxo upon deployment.	Medium	Addressed	The test passed, indicating that the contract logic did not fail, meaning the vulnerability is present in the current specification/implementation when locking, bidding and payout. This attack was simulated according to the failure mode when no NFT minting was associated.
SAA-0008	Upon deployment, Ada at produced utxo > `apMinBid`.	High	Addressed	The test confirms the expected failure mode, where the bidder receives the asset without providing any Ada, and the seller is penalized, because the bid was smaller than the value at the produced UTxO.
SAA-0009	Bidder only provides `newBid.bAmount - Ada` at consumed utxo	High	Addressed	Two tests were necessary for this requirement: a scenario with two bids and other with one bid and payout. Both tests confirm the failure, since the seller is penalized after both. In the first case, the bidder is refunded with more Ada than initially offered. In the second case, the bidder receives the asset by paying less Ada than `apMinBid`.
SAA-0010	Upon deployment: `apEndTime` < transaction's validity upper bound	Low	Addressed	Two tests were necessary for this requirement: the first certifies that no bid can be placed after `apEndTime`, and the second certifies that the

Reference	Failure Mode/Attack	Severity	Test Status	Comments
				seller can get the asset back on payout. The failure is confirmed, since one auction can be started with an end time set to the past.
SAA-0011	Upon bidding (`redeemer NewBid newBid`), the provided `AuctionDatum` inline datum is unbounded or `newBid.bAddr` has a `BuiltinByteString` > 32 bytes.	High	Addressed	The requirement necessitated two tests. In both instances, we set `newBid.bAddr` to a value greater than 32 bytes. The first test involved placing a bid followed by another bid, and the second proceeded with a payout after a bid. Both tests passed without a validation error, meaning the anticipated failure mode was not confirmed.
SAA-0012	Upon bidding (`redeemer NewBid newBid`), the new Auction UTxO is flooded with additional tokens.	High	Addressed	The testing focused on two scenarios, both involving an initial bid transaction with extra tokens. The first test, Multiple Bids, involved an initial bid with extra tokens followed by a second bid. This partially confirmed the expected failure mode: the second bid triggered an error indicating an overspent budget, when extra tokens are kept in the script. The second test, Bid and Payout, involved an initial bid with extra tokens followed by a payout. Contrary to expectations, the validator accepted the payout despite the presence of extra tokens.
SAA-0013	Seller deploys two (or more) Auction instances for the same Auctioned Token (Multiple Satisfaction/UTXO Contention).	High	Not Tested	This multiple-instance attack requires testing to ensure the NFT minting policy is correctly configured to issue only one

Reference	Failure Mode/Attack	Severity	Test Status	Comments
				unique NFT per auction instance.

Detailed SAA Analysis

This section provides an item-by-item analysis of each Security Assessment Artifact (SAA) failure mode, including the specific test outcome, detailed tester observations, and a placeholder for the relevant source code excerpts.

The preparation of parameters (`params`) utilized in the vast majority of tests was consistently the same. The standard configuration was designed to simulate a typical auction scenario while focusing on isolating the specific failure mode being tested.

```
let params =
       AuctionParams
         { apSeller = transPubKeyHash $ verificationKeyHash MockWallet.w1
         , apCurrencySymbol = cs
         , apTokenName = tokenName "NFT"
         , apMinBid = 10_000_000 -- 10 ADA minimum
         , apEndTime = POSIXTime 1640995220000 -- Slot 20
         }
```

For each test, a new Token is minted specifically for use in that test scenario.

```
let an = unTransAssetName (apTokenName params)
let mintTx = execBuildTx (BuildTx.mintPlutus mintingScript () an 1)
```

With the foundational setup complete, we can now proceed to the phase of implementing each specific test. The next sections detail the unit tests conducted for each individual SAA. In every test, we developed executable code to simulate potential vulnerabilities, ensuring coverage of described functionalities and edge-case scenarios that could be exploited by an attacker.

SAA-0001: Pre-set Highest Bid > Ada at Produced UTXO

Expected Failure Mode: First bidder can still make a formal offer if they bid more than `apMinBid` and provide the surplus Ada to refund the pre-set high bid, resulting in the Seller maliciously acquiring extra Ada.

Comments: As expected, without using a minting policy to mitigate this attack, the test passed. This happens, because the validator is not triggered during deployment. Thus, **the attack is confirmed**, and the Seller will maliciously acquire extra Ada.

The code below shows the preparation of a Fake Bid, which was used in the locking transaction.

Source Code Excerpt:

```
-- Create a fake bid to set as highest on deployment
 let fakeBidder = MockWallet.w1
     fakeBid =
       Bid
          { bAddr = PlutusTx.toBuiltin $ C.serialiseToRawBytes $ MockWallet.address
Defaults.networkId fakeBidder
         , bPkh = transPubKeyHash $ verificationKeyHash fakeBidder
         , bAmount = fromInteger $ C.unCoin value
         }

 -- Initial datum: no bids yet
 let initialDatum = AuctionDatum (Just fakeBid)
 let initialValue =
       BuildTx.assetValue
         (C.hashScript $ C.PlutusScript C.plutusScriptVersion mintingScript)
         an
         1

 -- Lock the NFT in the auction script
 let lockTx =
       execBuildTx $
          BuildTx.payToScriptInlineDatum Defaults.networkId scriptHash initialDatum
C.NoStakeAddress initialValue
           >> BuildTx.setMinAdaDepositAll Defaults.bundledProtocolParameters
```

SAA-0002: Pre-set Highest Bid < Ada at Produced UTXO

Expected Failure Mode: No one can make a formal offer, and PayOut will fail because the `newBid.bAmount` is less than the minimum Ada value required to transfer the Auctioned Token back to the seller, locking funds and the Auctioned Token forever.

Test Results: We created two tests. One for the bidding and one for payout. **The failure mode was not confirmed by the tests**, since the funds weren't locked when setting a fake small bid on deployment.

Comments on Bidding Test: A test was conducted where a fake bid of 0.85 ADAs was added in the Datum by the seller during the smart contract's deployment. This bidding amount was less than the `MinAda` required for the lock. Consequently, after the first legitimate bid, the seller received additional ADAs because the legitimate bidder was required to refund the fake bid amount to the seller.

Source Code Excerpt:

```
let bidTx =
       execBuildTx $ do
         BuildTx.addValidityRangeSlots 0 2
         -- Spend the script UTxO
         BuildTx.spendPlutusInlineDatum txIn (auctionValidatorScript params)
bidRedeemer
         -- Refund the fake bidder exactly the fake bid amount
         BuildTx.payToAddress (MockWallet.addressInEra Defaults.networkId
fakeBidder) (C.lovelaceToValue value)
         -- Pay to script with new datum and legitimate bid value
         BuildTx.payToScriptInlineDatum Defaults.networkId scriptHash newDatum
C.NoStakeAddress newValue
```

Comments on Payout Test: A test was conducted where a fake bid of 0.85 ADAs was added in the Datum by the seller during the smart contract's deployment. This bidding amount was less than the `MinAda` required for the lock. The payout transaction is only successfully processed when two steps are followed: first, a payment with the fake amount is made to the fake bidder, and second, the NFT is sent to the fake winner. However, this fake payment is never processed, i.e., the seller doesn't receive more ada. On the other hand, the contract will fail with the error "Not found: Output paid to seller" if the payment to the fake bidder is omitted.

Source Code Excerpt:

```
let payoutTx =
       execBuildTx $ do
         BuildTx.addValidityRangeSlots 20 22
         -- Spend the script UTxO with Payout redeemer
         BuildTx.spendPlutusInlineDatum txIn (auctionValidatorScript params)
payoutRedeemer
         -- Payout with a fake Bid on deployment, but without legitimate bids.
         -- Error happens when we comment this line.
         BuildTx.payToAddress (MockWallet.addressInEra Defaults.networkId seller)
                              (C.lovelaceToValue value)
         -- Pay the NFT to the highest bidder
         BuildTx.payToAddress (MockWallet.addressInEra Defaults.networkId seller)
lockedNftValue
```

SAA-0003: Pre-set Highest Bid = Ada at Produced UTXO

Expected Failure Mode: First bidder can still make a formal offer if they bid more than `apMinBid`, and the Seller is incorrectly refunded with the small `oldBid.bAmount` (e.g., min Ada value).

Test Results: As expected, without using a minting policy to mitigate this attack, the test passed. This happens, because the validator is not triggered during deployment. Thus, **the attack is confirmed**, and the Seller will maliciously acquire extra Ada.

Comments: The failure mode is present. The contract should strictly enforce `adHighestBid = Nothing` on initial deployment. By the end, the seller wallet will have the whole bid value, discounted the fees. The bidder will receive the asset, added with the minimum ADA required for the UTxO (not the whole locked ADA), discounted the fees. It means that the legitimate bidder is penalized.

Source Code Excerpt:

```
-- Create a fake bid to set as highest on deployment
 let fakeBidder = MockWallet.w1
     fakeBid =
       Bid
         { bAddr = PlutusTx.toBuiltin $ C.serialiseToRawBytes $ MockWallet.address
Defaults.networkId fakeBidder
         , bPkh = transPubKeyHash $ verificationKeyHash fakeBidder
         , bAmount = 3_000_000
         }

-- same value as the fake bid
let initialValue = BuildTx.assetValue policyId an 1 <> C.lovelaceToValue 3_000_000

-- Lock the NFT in the auction script + 3 ADAs (without `setMinAdaDepositAll`)
 let lockTx =
       execBuildTx $
         BuildTx.payToScriptInlineDatum Defaults.networkId scriptHash initialDatum
C.NoStakeAddress initialValue

let bidTx =
       execBuildTx $ do
         BuildTx.addValidityRangeSlots 0 2
         BuildTx.spendPlutusInlineDatum txIn (auctionValidatorScript params)
bidRedeemer
         -- Refund the fake bidder exactly the fake bid amount
         BuildTx.payToAddress (MockWallet.addressInEra Defaults.networkId
fakeBidder) (C.lovelaceToValue 3_000_000)
```

```
        BuildTx.payToScriptInlineDatum Defaults.networkId scriptHash bidDatum
C.NoStakeAddress bidNewValue

let payoutTx =
      execBuildTx $ do
        BuildTx.addValidityRangeSlots 20 22
        -- Spend the script UTxO with Payout redeemer
        BuildTx.spendPlutusInlineDatum txIn2 (auctionValidatorScript params)
payoutRedeemer
        -- Pay the bid amount to the seller
        BuildTx.payToAddress (MockWallet.addressInEra Defaults.networkId seller)
                             (C.lovelaceToValue 10_000_000)
        -- Pay the NFT to the highest bidder
        -- Notice that it's not being used the 'lockedNftValue', i.e., the bidder
        -- refunded the seller and will receive only the min ADA required
        BuildTx.payToAddress (MockWallet.addressInEra Defaults.networkId bidder)
nftOnly
          >> BuildTx.setMinAdaDepositAll Defaults.bundledProtocolParameters
```

SAA-0004: Unbounded `AuctionDatum` on Deployment (Extra Fields)

Failure Mode: Denial-of-Service (DoS) due to transaction size limits, or bidders being penalized by higher transaction fees.

Test Results: The tests indicated parsing error on Bid/PayOut, which **successfully mitigates the attack** by preventing the transaction from proceeding with an unbounded datum.

Comments: The validator's strict parsing of the datum structure effectively mitigates the Large Datum Attack on deployment.

Source Code Excerpt:

```
 let extraFields = replicate n (BI.mkI 42)

 -- Initial datum: no bids yet, but injects some extra fields on BuiltinData level
 let malformedDatum = BI.mkConstr 0 (PlutusTx.toBuiltinData (Nothing :: Maybe Bid)
: extraFields)

 -- Lock the NFT in the auction script with the malformed datum
 let lockTx =
       execBuildTx $
        BuildTx.payToScriptInlineDatum Defaults.networkId scriptHash malformedDatum
C.NoStakeAddress initialValue
          >> BuildTx.setMinAdaDepositAll Defaults.bundledProtocolParameters
```

SAA-0005: Large Reference Script on Deployment

Failure Mode: Denial-of-Service (DoS) due to transaction size limits, or bidders being penalized by higher transaction fees.

Test Results: Not Tested

Comments: Need to perform a test where a large reference script is explicitly attached to the Auction UTXO on deployment and try to execute a Bid/PayOut transaction. Need to understand better how sc-tools work with reference scripts. **A deeper comprehension of the interaction between `sc-tools` and reference scripts is necessary.**

SAA-0006: Auctioned Token Not Present on Deployment

Expected Failure Mode: No bids can be made, as the validation will fail. PayOut succeeds only if the Seller is able to manually provide the Auctioned Token, otherwise, funds are locked.

Test Results: Three distinct tests were conducted for this SAA:

1. **Bidding with No Auctioned Token:** The test failed, as expected, because the validator check for the presence of the auctioned token was effective. This indicates a successful mitigation for the bidding phase.
2. **PayOut with No Auctioned Token in Script UTXO, but in Seller's Wallet:** The test passed, meaning the payout transaction was successfully processed. This suggests a vulnerability where the auction could be concluded even if the token was never locked, as long as the seller could provide it during the payout transaction.
3. **PayOut with No Auctioned Token in Script UTXO and Not in Seller's Wallet:** The test resulted in an error, as anticipated, confirming that the contract fails if the token is completely absent from both the script UTXO and the seller's transaction inputs.

Comments: The failure mode was **confirmed**, without using a minting policy to mitigate this attack. While the contract prevents bids when the token is missing, the second test is problematic, since it allows payout when the token is present in the seller's wallet, but not locked in the script. The fact that the Payout can succeed without the token being explicitly locked in the script UTXO (as long as the seller provides it) suggests a flaw in the token *locking* requirement for a valid auction. The contract should strictly enforce that the Auctioned Token is present at the script UTXO upon deployment.

Source Code Excerpt: Bidding Test (Should Fail)

```
let initialValue = C.lovelaceToValue 2_000_000 -- Only ADA, no NFT

let lockTx =
       execBuildTx $
```

```
        BuildTx.payToScriptInlineDatum Defaults.networkId scriptHash initialDatum
C.NoStakeAddress initialValue
          >> BuildTx.setMinAdaDepositAll Defaults.bundledProtocolParameters

-- Create a transaction to place the bid
 let bidTx =
      execBuildTx $ do
        -- Set a validity range before auction end time
        BuildTx.addValidityRangeSlots 0 2
        -- Spend the script UTxO with redeemer (triggers the error)
        BuildTx.spendPlutusInlineDatum txIn (auctionValidatorScript params)
bidRedeemer
        -- Recreate script UTxO with updated datum and value
        BuildTx.payToScriptInlineDatum Defaults.networkId scriptHash newDatum
C.NoStakeAddress newValue
```

Source Code Excerpt: Payout Test (Token in Seller's Wallet - Success)

```
-- Create NFT in seller wallet (@TODO: create a minting policy)
 let mintTx = execBuildTx (BuildTx.mintPlutus mintingScript () an 1)

 -- Submit mint transaction from wallet 1 (the seller)
 _ <- tryBalanceAndSubmit mempty MockWallet.w1 mintTx TrailingChange []

let initialValue = C.lovelaceToValue 2_000_000 -- Only ADA, no NFT

let lockTx =
      execBuildTx $
        BuildTx.payToScriptInlineDatum Defaults.networkId scriptHash initialDatum
C.NoStakeAddress initialValue
          >> BuildTx.setMinAdaDepositAll Defaults.bundledProtocolParameters

let payoutTx =
      execBuildTx $ do
        BuildTx.addValidityRangeSlots 20 22
        -- Spend the script UTxO with Payout redeemer
        BuildTx.spendPlutusInlineDatum txIn (auctionValidatorScript params)
payoutRedeemer
        -- No bid - asset back to seller (but token is not present, so just ADA)
        BuildTx.payToAddress (MockWallet.addressInEra Defaults.networkId seller)
                             (C.lovelaceToValue 2_000_000)
```

Source Code Excerpt: Payout Test (Token Absent - Failure)

```
-- Don't create the NFT in the seller wallet, just proceed to lock an empty UTxO
with the auction script
let initialValue = C.lovelaceToValue 2_000_000

let lockTx =
       execBuildTx $
         BuildTx.payToScriptInlineDatum Defaults.networkId scriptHash initialDatum
C.NoStakeAddress initialValue
           >> BuildTx.setMinAdaDepositAll Defaults.bundledProtocolParameters

let payoutTx =
       execBuildTx $ do
         BuildTx.addValidityRangeSlots 20 22
         -- Spend the script UTxO with Payout redeemer
         BuildTx.spendPlutusInlineDatum txIn (auctionValidatorScript params)
payoutRedeemer
         -- No bid - asset back to seller (but token is not present at UTxO and
seller's wallet, so just ADA)
         BuildTx.payToAddress (MockWallet.addressInEra Defaults.networkId seller)
                              (C.lovelaceToValue 2_000_000)
```

SAA-0007: Other Tokens Present on Deployment

Failure Mode: First Bidder can acquire extra tokens when making a formal offer, penalizing the Seller.

Test Results: The test passed as `OK`, meaning the contract **allows** locking, bidding, and payout with extra tokens. This **confirms the failure** investigated by this test case.

Tester Comments: The contract is vulnerable to token leakage, when no NFT minting policy is used as a mitigation.

Source Code Excerpt:

```
 let an = unTransAssetName (apTokenName params)
     ot1 = C.UnsafeAssetName "Other Token 1"
     ot2 = C.UnsafeAssetName "Other Token 2"

 -- Mint NFT + other tokens
 let mintTx = execBuildTx $ do
               BuildTx.mintPlutus mintingScript () an 1
               BuildTx.mintPlutus mintingScript () ot1 10
```

```
                BuildTx.mintPlutus mintingScript () ot2 20

 let initialValue = BuildTx.assetValue policyId an 1
                      <> BuildTx.assetValue policyId ot1 10
                      <> BuildTx.assetValue policyId ot2 20
```

SAA-0008: Ada at Produced UTXO > `apMinBid` on Deployment

Expected Failure Mode: Bidder can place a bid with less Ada than what is available in the produced UTxO. Seller is penalized upon PayOut. This is a critical error because the amount of Ada locked with the initial UTXO sets the *actual* minimum value that the next bidder must refund to the previous (initial) bid, regardless of `apMinBid`. If the locked Ada is greater than `apMinBid`, the contract logic could be bypassed, allowing a bidder to effectively acquire the asset without matching the true economic minimum.

Test Results: The test passed, confirming that the deployment is correctly blocked when the Ada at the produced UTXO is greater than `apMinBid`. **The failure mode was confirmed by the tests, since the seller is indeed penalized on these conditions.**

Comments: The validator doesn't prevent the auction from being deployed with an initial Ada value in the UTXO that exceeds the specified `apMinBid`. This allows a scenario where a bidder exploits the logic by bidding less than the inflated locked amount but more than `apMinBid`, which leads to asset acquisition without appropriate compensation for the seller.

Source Code Excerpt:

```
  -- Lock NFT in script
 let initialDatum = AuctionDatum Nothing
 let initialValue = BuildTx.assetValue policyId an 1
                      <> C.lovelaceToValue 15_000_000 -- More ADA than the minimum
bid value
 -- The new bid will be exactly the minimum
 let bidNewValue = nftOnly <> C.lovelaceToValue 10_000_000

 -- The seller will receive only the minimum bid amount
 let payoutTx =
       execBuildTx $ do
         BuildTx.addValidityRangeSlots 20 22
         -- Spend the script UTxO with Payout redeemer
         BuildTx.spendPlutusInlineDatum txIn2 (auctionValidatorScript params)
payoutRedeemer
         -- Pay the bid amount to the seller
         BuildTx.payToAddress (MockWallet.addressInEra Defaults.networkId seller)
```

```
                                (C.lovelaceToValue 10_000_000)
        -- Pay the NFT to the highest bidder
        BuildTx.payToAddress (MockWallet.addressInEra Defaults.networkId bidder)
nftOnly
          >> BuildTx.setMinAdaDepositAll Defaults.bundledProtocolParameters
```

SAA-0009: Ada at Produced UTXO is High but `< apMinBid`

Expected Failure Mode: First bidder is refunded with more Ada than expected when a second Bid is placed, and Seller is penalized upon PayOut.

Test Results: Two tests were conducted for this requirement: a scenario with two bids and another with one bid followed by payout. Both tests confirm the failure mode, as the contract logic allows the seller to be penalized after both scenarios. This indicates that **the failure mode was confirmed by the tests**.

Comments: The contract logic failed to enforce the correct refund mechanism. In the two-bid scenario, the first bidder was refunded with more Ada than initially offered, effectively exploiting the contract. In the bid and payout scenario, the bidder received the auctioned asset while paying less Ada than the specified `apMinBid`, resulting in a financial loss for the seller.

Source Code Excerpt: Bidding Twice Test

```
 -- A high value locked in the script, but less than apMinBid
 let initialValue = nftOnly <> C.lovelaceToValue 8_000_000

 let bidder1 = MockWallet.w2
     firstBid =
       Bid
         { bAddr = PlutusTx.toBuiltin $ C.serialiseToRawBytes $ MockWallet.address
Defaults.networkId bidder1
         , bPkh = transPubKeyHash $ verificationKeyHash bidder1
         , bAmount = fromInteger $ C.unCoin 10_000_000
         }
     bid1Redeemer = NewBid firstBid
     bid1Datum = AuctionDatum (Just firstBid)

 -- The script will have NFT + Bid 2
 let bid2NewValue = nftOnly <> C.lovelaceToValue 15_000_000
 -- Refund all 10 ADAs to the first bidder
 let bid1RefundValue = C.lovelaceToValue 10_000_000
```

Source Code Excerpt: Bidding and Payout

```
 -- Seller should receive locked value, but not the NFT
 let sellerPayoutValue = C.lovelaceToValue 10_000_000
```

```
 let payoutTx =
       execBuildTx $ do
         BuildTx.addValidityRangeSlots 20 22
         -- Spend the script UTxO with Payout redeemer
         BuildTx.spendPlutusInlineDatum txIn1 (auctionValidatorScript params)
payoutRedeemer
         -- Pay 10 ADAs to the seller (but 8 ADAs were already locked)
         BuildTx.payToAddress (MockWallet.addressInEra Defaults.networkId seller)
sellerPayoutValue
         -- Pay the NFT to the highest bidder (with min ADA required)
         BuildTx.payToAddress (MockWallet.addressInEra Defaults.networkId bidder1)
nftOnly
           >> BuildTx.setMinAdaDepositAll Defaults.bundledProtocolParameters
```

SAA-0010: `apEndTime` < Transaction Validity Upper Bound on Deployment

Failure Mode: No bids can be made, but Seller can get the Auctioned Token back via PayOut. The core vulnerability is the ability to deploy an auction with an `apEndTime` already in the past.

Test Results: The tests passed as OK, confirming that the contract correctly enforces the deadline during deployment, bidding, and payout stages when testing the scenarios. **The failure is confirmed** because the tests show that an auction can be successfully deployed with an end time set to the past.

Comments: The contract logic should prevent an auction from being deployed if `apEndTime` is less than the current time (or the transaction's validity upper bound). The confirmed failure means a seller can start an auction that is immediately expired. The subsequent tests confirmed the expected behavior of an expired auction:

1. **Bidding Test:** Certifies that no bid can be placed after `apEndTime`.
2. **Payout Test:** Certifies that the seller can get the asset back on payout if the auction is deployed as expired.

Source Code Excerpt: Bidding After End Time (Should Fail)

```
 -- Set the slot to a time after auction end
 setSlot slot

 -- Create a transaction to place the bid
 let bidTx =
       execBuildTx $ do
         -- Set a validity range before auction end time
```

```
        BuildTx.addValidityRangeSlots slot (slot + 1)
        -- Spend the script UTxO with redeemer
        BuildTx.spendPlutusInlineDatum txIn (auctionValidatorScript params)
bidRedeemer
        -- Recreate script UTxO with updated datum and value
        BuildTx.payToScriptInlineDatum Defaults.networkId scriptHash newDatum
C.NoStakeAddress newValue
```

Source Code Excerpt: Payout After End Time (Should Succeed)

```
setSlot 30 -- The auction ended at slot 20

 let payoutRedeemer = Payout
     seller = MockWallet.w1

 let payoutTx =
       execBuildTx $ do
        BuildTx.addValidityRangeSlots 30 32
        -- Spend the script UTxO with Payout redeemer
        BuildTx.spendPlutusInlineDatum txIn (auctionValidatorScript params)
payoutRedeemer
        -- Payout with no bid - asset back to seller
        BuildTx.payToAddress (MockWallet.addressInEra Defaults.networkId seller)
                             lockedNftValue
```

SAA-0011: Unbounded `newBid.bAddr` on Bidding

SAA-0011: Unbounded `newBid.bAddr` on Bidding

Failure Mode: Large Datum Attack where `newBid.bAddr` size exceeds limits, causing a DoS for subsequent Bid transactions.

Test Results: The tests passed without a validation error, meaning the anticipated failure mode **was not confirmed**. The only way we received a validation error was when we created an extremely large value for `newBid.bAddr`, exceeding the size limits of the UTxO itself.

Comments: The requirement necessitated two tests: placing a bid followed by another bid, and a bid followed by a payout. In both instances, we set `newBid.bAddr` to a value exceeding 32 bytes (the typical size limit for addresses/hashes). Both tests passed, which confirms the contract's vulnerability to the Large Datum Attack on bidding. The vulnerability is present because the validator does not strictly check the size of `newBid.bAddr`, allowing an attacker to inflate the datum size, which could lead to Denial-of-Service (DoS) for subsequent transactions by exceeding script budget limits.

Source Code Excerpt for both test cases:

```
let bidderAddrAttack = foldl PlutusTx.appendByteString PlutusTx.emptyByteString
(replicate 154 bidderAddr)

let firstBid =
       Bid
         { bAddr = bidderAddrAttack <> "XXXXXXXXXXXXXX" -- Use the attack address
that exceeds 32 bytes (here we push to the UTxO's size limit)
         , bPkh = transPubKeyHash $ verificationKeyHash bidder1
         , bAmount = 30_000_000
         }
```

SAA-0012: UTXO Flooded with Additional Tokens on Bidding

Failure Mode: Large Value Attack/DoS where the script budget is exceeded when searching for the required tokens in the Value map, locking funds forever.

Test Results: The testing focused on two scenarios, both involving an initial bid transaction with extra tokens.

1. **Multiple Bids Test:** An initial bid with extra tokens was followed by a second bid. This partially confirmed the failure mode: the second bid triggered an error indicating an overspent budget, suggesting a Denial-of-Service (DoS) vulnerability, when the tokens are kept locked in the script during the second bid. If the extra tokens are refunded to the first bidder, the transaction performs normally. Triggering this error required adjusting the Unit steps limit.
2. **Bid and Payout Test:** An initial bid with extra tokens was followed by a payout. Contrary to expectations, the validator accepted the payout despite the presence of extra tokens.

The failure mode is partially confirmed. While the Multiple Bids test suggests a DoS vulnerability, the Payout test demonstrates a flaw in the validator's token check, allowing extra tokens to remain in the script UTXO after payout.

Comments: The contract is vulnerable to a Large Value Attack and potential token leakage. The mitigation to ensure *only* the Auction NFT and the highest bid Ada are present at the produced UTXO upon a new bid is not implemented in the validator logic.

Source Code Excerpt: Multiple Bids Test

```
 -- First Bid Information

 -- Generate MANY garbage token names under same policy
 let garbageTokenNames =
```

```
      fmap
        (\i -> C.UnsafeAssetName (pack $ "A" <> show i))
        ([1 .. 181] :: [Int]) -- budget limit (had to change Defaults to reach
this limit)

 -- Mint all garbage tokens
 let mintGarbageTx =
      execBuildTx $ do
        forM_ garbageTokenNames $ \tn ->
          BuildTx.mintPlutus mintingScript () tn 1

 -- Build flood value (all under SAME policyId)
 let garbageValue =
      mconcat $
        fmap
          (\tn -> BuildTx.assetValue policyId tn 1)
          garbageTokenNames

 -- Garbage value + Locked Tokens + Bid Value
 let bid1NewValue = garbageValue <>
                    tokensOnly <>
                    C.lovelaceToValue 30_000_000

-- Second Bid Information: triggers the error

 let bid2NewValue = garbageValue <> nftOnly <> C.lovelaceToValue 50_000_000 -- The
script will have Extra Tokens + NFT + Bid 2
 let bid1RefundValue = C.lovelaceToValue 30_000_000 -- refunds the locked value,
except NFT and Extra Tokens

-- Second Bid Information: works normally

let bid2NewValue = nftOnly <> C.lovelaceToValue 50_000_000 -- The script will have
NFT + Bid 2
 let bid1RefundValue = garbageValue <> C.lovelaceToValue 30_000_000 -- refunds
everything, except NFT
```

Source Code Excerpt: Bid and Payout Test

```
-- Payout Information

let sellerPayoutValue = garbageValue <> C.lovelaceToValue 30_000_000 -- Seller
should receive extra tokens + locked value, but not the NFT
```

```
let payoutTx =
       execBuildTx $ do
         BuildTx.addValidityRangeSlots 20 22
         -- Spend the script UTxO with Payout redeemer
         BuildTx.spendPlutusInlineDatum txIn1 (auctionValidatorScript params)
payoutRedeemer
         -- Pay the bid amount to the seller + extra tokens
         BuildTx.payToAddress (MockWallet.addressInEra Defaults.networkId seller)
sellerPayoutValue
         -- Pay the NFT to the highest bidder + min ada
         BuildTx.payToAddress (MockWallet.addressInEra Defaults.networkId bidder1)
lockedNftValue
```

SAA-0013: Multiple Auction Instances for the Same Auctioned Token

Failure Mode: UTXO Contention/Multiple Satisfaction where a Bidder consumes multiple Auction UTXOs in one transaction, getting refunded multiple times and acquiring the Auctioned Token and funds maliciously.

Test Results: Not Tested

Comments: This requires testing the NFT minting policy to confirm it enforces the uniqueness of the Auction NFT token, preventing multiple valid auction instances for the same asset.

Final Remarks

The security audit of the smart contract revealed several critical vulnerabilities, with the majority of confirmed high-severity exploits stemming from a lack of validation during the initial contract deployment, known as the locking phase. This omission allows a malicious seller to execute various attacks (SAA-0001, SAA-0003, SAA-0007, SAA-0008, SAA-0009, SAA-0010). These exploits collectively allow the seller to acquire extra Ada unfairly, lock an auction that has already expired, or initialize the auction with incorrect funds, thereby penalizing bidders or causing direct loss of tokens/funds for the seller.

Further confirmed issues relate to inadequate token handling and potential Denial-of-Service (DoS) vectors. The contract exhibits a major flaw (SAA-0006) where the auction can complete its payout process even if the auctioned token was not explicitly locked within the script UTXO initially, as long as the seller provides the token later. This fundamentally violates the security requirements of an auction setup. Furthermore, partial vulnerabilities were confirmed (SAA-0012) related to bidders flooding the UTXO with extra tokens, leading to budget overruns and subsequent DoS errors for later bidders. While testing for unbounded addresses (SAA-0011) did not confirm a direct validation

failure, it demonstrated the contract's susceptibility by requiring large input values to trigger existing limitations, indicating a continued risk of large datum attacks.

It is noteworthy that some expected vulnerabilities were not confirmed, while others were successfully mitigated. Specifically, tests for SAA-0002 (Pre-set Highest Bid < Ada at Produced UTXO) did not lead to the anticipated outcome of permanently locking funds but revealed an unexpected mechanism where the seller still managed to receive additional Ada from the legitimate bidder. Conversely, the vulnerability related to unbounded `AuctionDatum` (SAA-0004) was effectively prevented by the validator's strict parsing rules.

Crucially, two high-severity attacks remain in a **Not Tested** status: SAA-0005 (Large Reference Script) and SAA-0013 (Multiple Auction Instances). The mitigation strategy for the latter, which involves UTXO Contention, is entirely dependent on a correctly implemented NFT minting policy to ensure token uniqueness, which necessitates dedicated testing. The inability to fully test SAA-0005 is currently constrained by technical limitations in understanding how `sc-tools` interacts with excessively large reference scripts.

To mitigate the confirmed risks, the most critical step is addressing the absence of validation in the locking transaction. A dedicated validation check for the deployment transaction is essential (by using a NFT minting policy associated to the Auction Script), as it directly mitigates the majority of the confirmed high-severity exploits (SAA-0001, SAA-0003, SAA-0007, SAA-0008, SAA-0009, SAA-0010). Additionally, the contract must be strictly modified to enforce that *only* the Auctioned Token and the required minimum Ada are present in the script UTXO across all transitions (Locking, Bidding, and Payout). This strict token check is essential to prevent token leakage and related vulnerabilities (SAA-0006, SAA-0007, SAA-0012).